STAR GAMER

掌上游戏机用户手册

NES 模拟器 · Arduboy · 音乐 · 图片 · 文本 · 视频 · AXF 应用

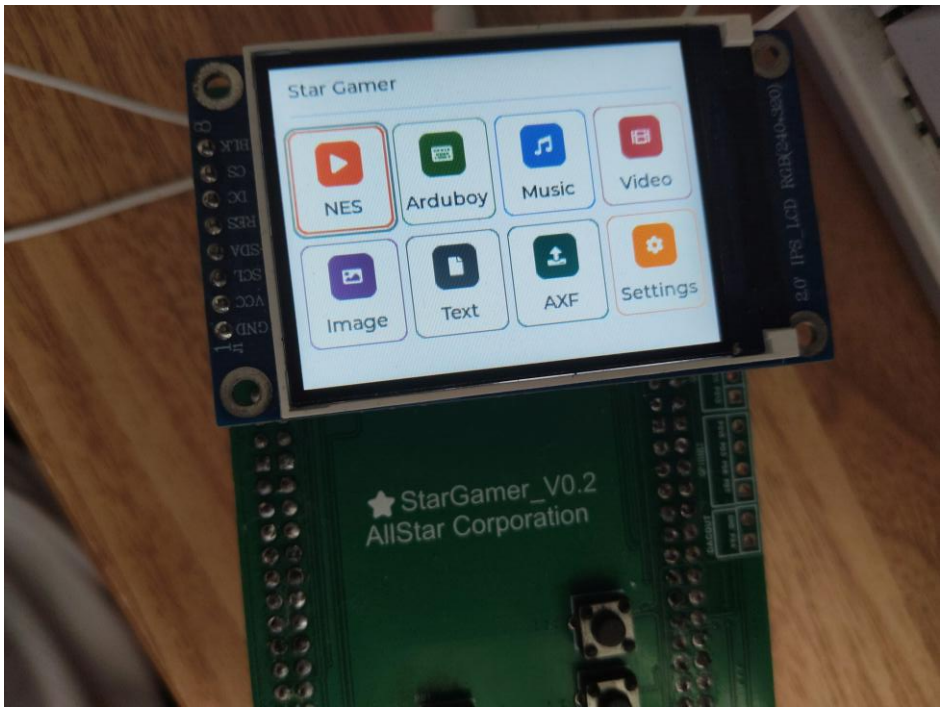


图 1　实机主菜单（以当前固件页面为准）

> **阅读提示** 本手册依据当前工程代码、设置页面和实机素材编写。旧版 Docs 文档可能与当前固件不同；遇到差异时，以设备屏幕和本手册的“当前固件行为”描述为准。

目标硬件：STM32F407ZGT6 · ST7789 320×240 · PCM5102 · W25Q64 · AT24C02 · IS62WV51216HBLL

适用版本：Star Gamer 1.1.3-preview-oc（Clock profile 可在设置中选择）

目录

本手册按用户操作、二次开发和系统架构三部分组织。目录由标题样式自动生成，也可使用 Word 导航窗格（视图 → 导航窗格）按标题层级快速跳转。

第一部分　快速上手与功能说明 4

1　设备、按键与启动流程 4

1.1　按键总表 4

1.2　硬件平台技术参数 5

1.3　时钟档位与主频切换 5

2　主菜单与文件浏览 6

2.1　支持的文件类型 6

2.2　文件浏览器的实现细节 7

3　NES / Arduboy 游戏 7

3.1　NES 7

3.2　Arduboy 9

4　音乐、视频、图片与文本 10

4.1　音乐播放器 10

4.2　视频播放器 11

4.3　图片查看器 12

4.4　文本查看器 12

5　AXF 动态应用 13

6　设置、字库、USB MSC 与固件升级 14

6.1　设置页 14

6.2　安装字库 14

6.3　USB MSC 与升级 15

第二部分　二次开发指导 16

7 UART Shell 命令 16
7.1 诊断命令使用建议 17
8 Shell 命令导出 17
9 固件 API 使用约定 18
9.1 内存分配与线程创建规范 18
10 AXF API 使用 19
10.1 SVC 调用机制与资源模型 20
第三部分 系统架构 21
11 启动链路与内存/存储框架 21
11.1 存储介质映射 22
12 SDIO 防御性队列 23
13 GUI 管理器与 LVGL-LCD 握手 24
13.1 显示所有权握手 25
14 BootLoader + IAP 26
14.1 升级状态机与断电恢复 27
15 AXF 加载器架构 27
16 视频播放器、LibJPEG 与 LCD 流水线 28
第四部分 性能、诊断与附录 30
17 视频性能参考报告 30
17.1 如何读 videostat 30
17.2 两组实测数据对比 31
17.3 选片与转码建议 31
18 故障排查与诊断 32
18.1 HardFault 与内存转储 32
附录 A 构建、目录与支持范围 33

提示：页码以实际排版为准，更新域后自动刷新。

第一部分　快速上手与功能说明

1　设备、按键与启动流程

Star Gamer 是一台以 STM32F407ZGT6 为核心的掌上游戏机。显示使用 ST7789 320×240 RGB565 SPI 屏，音频由 I2S3 + DMA 输出到外部 PCM5102；SD 卡通过 SDIO/FatFs 提供游戏和媒体文件，W25Q64 保存字库、升级暂存和诊断转储，AT24C02 保存设置与升级记录。

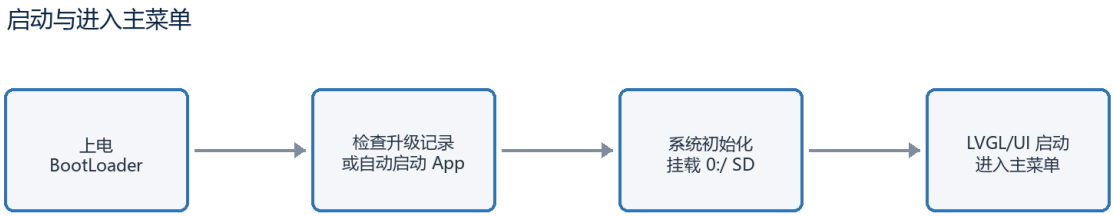


图 2　启动路径：BootLoader 负责更新与跳转，应用负责外设和 GUI 初始化。

正常开机时，BootLoader 自动检查升级状态并跳转到应用；按住 PB7（Select/Exit）开机可进入 BootLoader 菜单。应用初始化 SD 卡、Shell（若启用）、资源表和 GUI 任务，启动页显示 Starting GUI... 后进入主菜单。

1.1　按键总表

按键	GPIO	常规 GUI	游戏/媒体别名
Key1	PB1	上移	UP
Key2	PC1	左移/上一页	LEFT
Key3	PA2	右移/下一页	RIGHT
Key4	PA1	下移	DOWN
Key5	PE2	确认/Enter	OK / Start

按键	GPIO	常规 GUI	游戏/媒体别名
Key6	PE3	应用动作	A
Key7	PB7	返回/Esc	Select / Exit
Key8	PG14	应用动作	B

导航约定 方向键移动焦点；OK 确认；B 或 Select 返回。文件浏览器中 Left/Esc 返回上级，Up/Down 选择，Right 或 Enter 打开目录或文件。主菜单按 4 列循环移动。

1.2 硬件平台技术参数

Star Gamer 的硬件平台围绕 STM32F407ZGT6 构建，板载 192 KB 片内 SRAM 并外扩 1 MB SRAM。下表汇总与日常使用和二次开发最相关的关键参数。

项目	规格
主控	STM32F407ZGT6（Cortex-M4F，1 MB Flash）
主频档位	192 / 216 / 240 MHz 三档，设置页切换，默认 240 MHz
内存	128 KB 主 SRAM + 64 KB CCM（CPU 专用）+ 1 MB 外部 SRAM（FSMC）
显示	ST7789 320×240 RGB565，SPI + DMA 传输，背光 PWM 调光
音频	I2S3 + DMA 双缓冲输出至 PCM5102 DAC
存储	SD 卡（SDIO 4-bit + DMA）、W25Q64 8 MB SPI Flash、AT24C02 256 B EEPROM
USB	OTG FS，MSC 设备模式（SD 卡模拟 U 盘）
调试	UART1 控制台（Letter Shell），printf 重定向

1.3 时钟档位与主频切换

系统时钟由 8 MHz 外部晶振经 PLL 倍频得到。固件提供 192/216/240 MHz 三档：默认 240 MHz 保证视频解码与 GUI 流畅度；对续航或发热敏感的场景可切换到低速档。

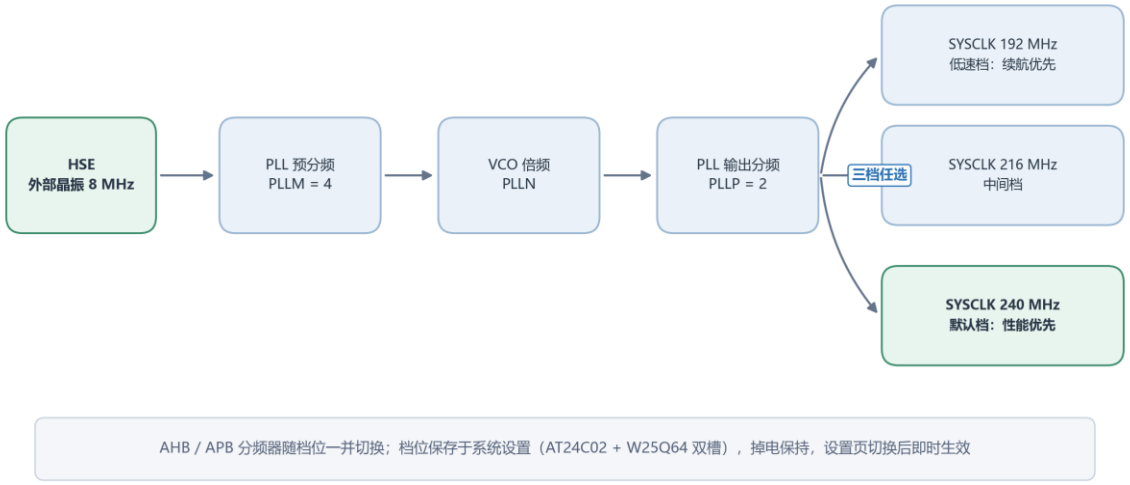


图3　系统时钟树与三档主频。

档位在设置页切换，选择后立即保存到系统设置并提示复位生效（Clock saved; reset MCU to apply）；重启后固件按新档位重配 PLL 与总线分频。档位与音量、主题等设置一同持久化，掉电不丢失（见 6.1 节）。

2　主菜单与文件浏览

当前主菜单包含 NES、Arduboy、Music、Video、Image、Text、AXF 和 Settings 八项。除 Arduboy 外，媒体和应用页面通常从 SD 卡根目录 0:/ 开始浏览，并按扩展名过滤。文件名可为 UTF-8 或 OEM/GBK，显示层会尽力转换；乱码时优先检查文件名编码。

2.1　支持的文件类型

页面	扩展名/来源	当前行为
NES	.nes	从 0:/ 选择 ROM，启动模拟器；退出后恢复 GUI。
Arduboy	内置列表	当前注册 Buttons、Dino、FlappyBall，最多支持 8 个游戏。
Music	.wav / .mp3	WAV PCM 或 Helix MP3；播放、暂停、停止、音量和 Seek。

页面	扩展名/来源	当前行为
Video	.avi	MJPEG 视频 + PCM 音频，最大 320×240；支持暂停、停止、Seek、音量。
Image	.jpg / .jpeg	LibJPEG 异步解码后查看。
Text	.txt .log .ini .md .c .h .cpp .hpp .json .xml	UTF-8/GBK/BOM 检测，分页与滚动查看。
AXF	.axf	动态加载可重定位应用，PB7 请求退出。
Font	.bin / .xbf	安装到 W25Q64 字库区，供 Text/LVGL 使用。

2.2 文件浏览器的实现细节

所有媒体与应用页面共用同一个文件浏览器组件（GUI_FileManage），行为一致：

- 按页面声明的扩展名过滤，目录排在文件之前；媒体页面从 SD 卡根目录 0:/ 开始浏览。
- 文件名支持 OEM（GBK）与 UTF-8 双编码，界面侧完成转码，中文与长文件名不乱码。
- 列表采用异步分页加载，单页最多 32 项（GUI_FILEMANAGE_MAX_ITEMS），大目录不会阻塞界面。
- 浏览器上下文从 SRAMEX 动态分配，进入页面时创建、退出时归还，降低主 SRAM 占用。

3 NES / Arduboy 游戏

3.1 NES

进入 NES 后选择根目录中的 .nes 文件。启动前 GUI 会暂停并释放 LCD/SPI 所有权，模拟器使用主 SRAM 静态条带缓冲向屏幕输出；退出时释放运行时缓冲并恢复 GUI。NES 别名按键为 Up=PB1、Down=PA1、 Left=PC1、 Right=PA2、 Start=PE2、 A=PE3、 B=PG14、 Exit=PB7。

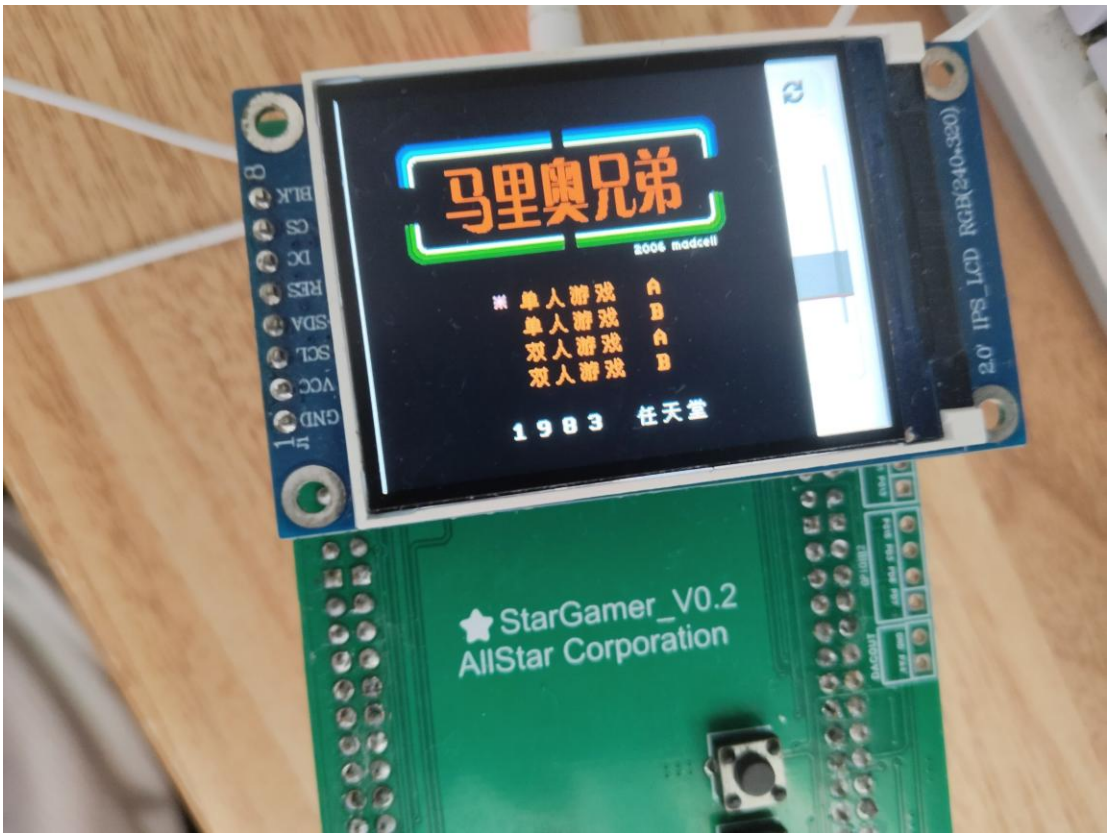


图4　NES 文件选择/运行界面。

异常提示 NES file error 表示文件读取或格式问题；Unsupported mapper 表示当前核心不支持该映射器；NES memory error 表示运行时缓冲申请失败。

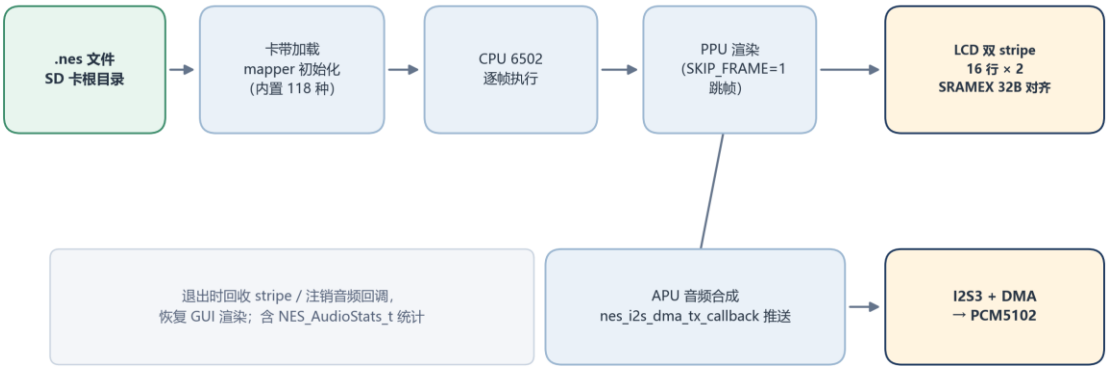


图5　NES 模拟器运行时数据流：视频经双 stripe 直写 LCD，音频走系统 I2S 回调链。

模拟器与硬件协同的几个关键实现：

- 跳帧策略 NES_SKIP_FRAME=1：每两帧渲染一次，把 CPU 预算让给 PPU/APU 时序。
- 渲染输出使用 16 行 × 2 的 LCD 双 stripe，缓冲为主 SRAM 中的静态数组。
- APU 音频经 audio_card_register_tx_index(nes_i2s_dma_tx_callback) 挂入系统 I2S 回调链，与音乐/视频共享同一声卡架构。
- 内置 118 个 mapper（含常见 MMC1/MMC3 等），不支持的卡带会在加载时提示。
- 提供 NES_AudioStats_t 统计（回调/欠载计数），便于定位声音断续问题。

3.2 Arduboy

Arduboy 页面直接列出固件内置游戏。Arduboy 兼容层提供 128×64 单色帧缓冲和按键适配，运行时按方向键、A/B 操作，PB7 退出。退出后页面生命周期由 Arduboy_Runtime 统一收尾，避免残留线程或 LCD 状态。

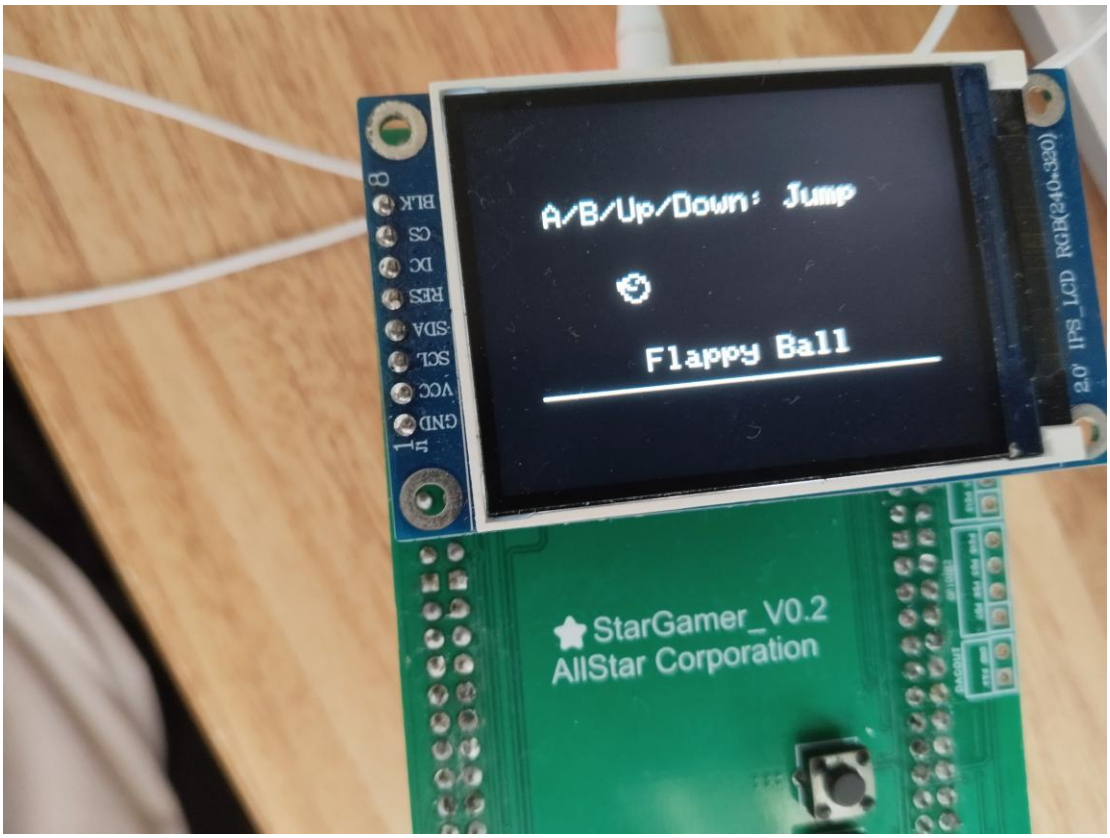


图6 Arduboy 游戏列表界面。

4 音乐、视频、图片与文本

4.1 音乐播放器

Music 页面支持 .wav 和 .mp3。播放器核心通过 codec 接口统一处理读取、Seek、时长和关闭；MP3 使用 Helix 定点解码器。播放中不能直接打开另一个文件，先按 Stop 再重新选择。进度条使用 0..1000 的千分比表示，左右键每次移动约 2.5%，按 OK 提交 Seek；上下键调节音量，范围 0..100。

操作	按键/控件	说明
播放/暂停	OK 或按钮	切换当前文件的播放状态。
停止	B/Stop	关闭 codec，清空音频管道。
Seek	左右 + OK	左右移动位置，OK 确认。
音量	上下	0..100，设置页默认 70。

播放器核心（Player.c）把解码器抽象为统一的 codec 接口（read_pcm / seek / tell_seconds / close）：WAV 直接解析 PCM 数据块，MP3 由 Helix 定点库以 5120 B 窗口解码。解码出的 PCM 帧进入播放管道，由声卡回调推入 I2S DMA 双缓冲连续输出；进度与时长来自解码位置回调，Seek 在解码器内部完成文件定位。



图 7 音乐播放数据流：解码管道直通声卡回调，格式差异被 codec 接口隔离。

4.2 视频播放器

Video 页面只接受当前 AVI 管线支持的 MJPEG 视频和 PCM 音频，解析器限制画面不超过 320×240。播放期间 Video Task 调用 lvgl_stop_ui()，等待 LVGL 确认暂停后独占 SPI1/LCD；退出时调用 lvgl_start_ui()。OK 暂停/恢复，B 或 Exit 停止，上下调音量，左右以 5% 步长 Seek。音频时钟作为同步基准，视频晚到时允许丢帧，优先保持声音连续。

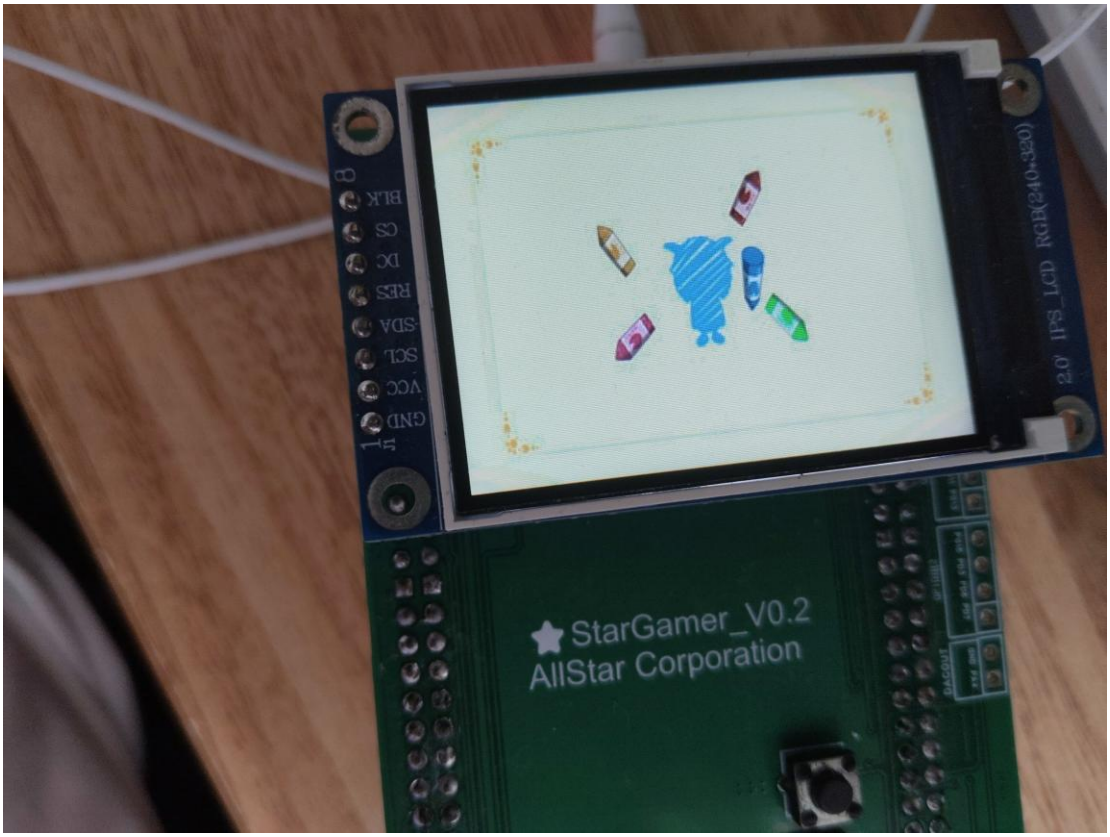


图 8 AVI 视频播放器界面。

当前 AVI 管线只接受 MJPEG 视频轨 + PCM 音频轨的组合，分辨率上限 320×240，不满足的文件会在解析阶段被直接拒绝。制作或转码视频时建议：

项目	建议值
容器	AVI；解析器打开时建立流索引，Seek 基于索引与 chunk 边界容错实现
视频	MJPEG，≤320×240，23.976 fps 或 25 fps
视频码率	≤2 Mbps 可全帧率流畅播放；10 Mbps 仍可播放，但会持续丢帧（见 17 章实测）

项目	建议值
音频	PCM 22050 Hz / 16 bit，建议立体声，与视频交错存储
SD 卡	优先选择高速卡；读取吞吐不足先表现为丢帧增多，而不是声音卡顿

4.3 图片查看器

Image 页面选择 .jpg/.jpeg 后异步解码，避免在文件浏览器线程中长时间阻塞。Viewer 中按 OK、Esc 或 Left 返回文件列表；LibJPEG 解码上下文和 arena 走默认 SRAMEX 堆分配，显示条带和 DMA 缓冲同样使用可 DMA 的 SRAMEX。

4.4 文本查看器

Text 页面支持常见文本、源码和配置扩展名，自动检测 UTF-8、GBK 和 BOM。上一页/下一页按钮对应 Left/Right，Up/Down 滚动，滚动到边界时自动翻页。缺少字形时使用 fallback 字符，未安装字库会提示 Install font first。



图9　文本查看器的分页与滚动界面。

5 AXF 动态应用

AXF Loader 从 0:/ 选择 .axf，调用 dl_load_file() 装载可重定位代码并取得 entry，随后创建应用线程和监控线程。Settings 中的 AXF stack 可选 4/8/12/16/24 KB，默认 16 KB。加载页会显示 PB7 exits AXF；按 PB7 发出 cooperative exit，监督线程最多等待 100 ms，应用不返回时才执行强制收尾。

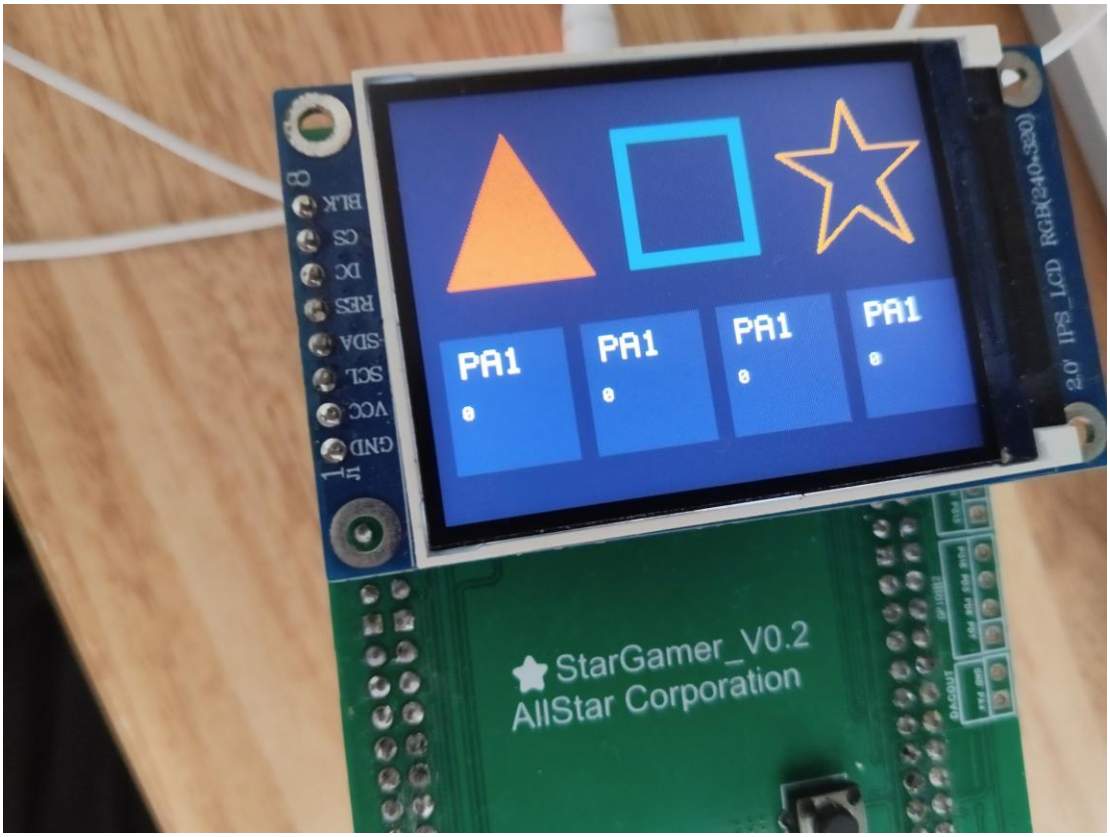


图 10　AXF 动态应用演示。

AXF 约束 AXF 应用必须成对调用 SG_LCD_Acquire()/SG_LCD_Release()，循环中轮询 SG_ShouldExit()。PB7 是宿主保留的退出键；应用不轮询或不释放 LCD 会增加监督线程强制结束的概率。

加载器把一次运行抽象为 IDLE → LOADING → STARTING → RUNNING → STOPPING 的状态机：选择文件后由 dl_load_file() 完成 ELF 解析与重定位；随后按 Settings 中的 AXF stack 档位（4/8/12/16/24 KB）分配应用栈，创建应用线程与监控线程；运行中按 PB7（Select/Exit）或应用自行返回进入 STOPPING，监控线程回收加载段与栈后回到 IDLE。任何阶段失败都会进入 ERROR 并提示原因，已分配的资源同样被回收。架构细节见第 15 章。

6 设置、字库、USB MSC 与固件升级

6.1 设置页

分组	可选项/默认值	生效方式
Theme	深色（默认）/浅色	立即异步重载 Settings 页面。
Clock	192 / 216 / 240 MHz（默认 240）	保存后提示 reset MCU to apply。超频配置需确认硬件和散热。
ScreenLight	0..100（默认 100）	背光 PWM。
音量	0..100（默认 70）	播放器和音频管线使用。
AXF stack	4/8/12/16/24 KB（默认 16）	下次启动 AXF 时使用。
USB MSC	关闭（默认）/开启	应用 USB MSC 设置；开启后 SD 卡作为 U 盘。
Debug	Debug、Shell、Audio/Video/LibJPEG/FatFs Log	Debug/Shell 改动需重启。
Memory Dump	总开关及 CCM/Main SRAM/SRAMEX 子开关	HardFault 时写入 W25Q64。

6.2 安装字库

1. 在 SD 卡根目录准备 .bin 或 .xbf 字库文件。
2. 进入 Settings → Install Font，选择文件并确认。
3. 固件会校验格式，擦除 W25Q64 字库区（起始地址 0，预留 2 MB），写入后回读验证并保存设置。
4. 重新进入 Text 页面；若仍提示 Install font first，检查文件格式和写入验证结果。

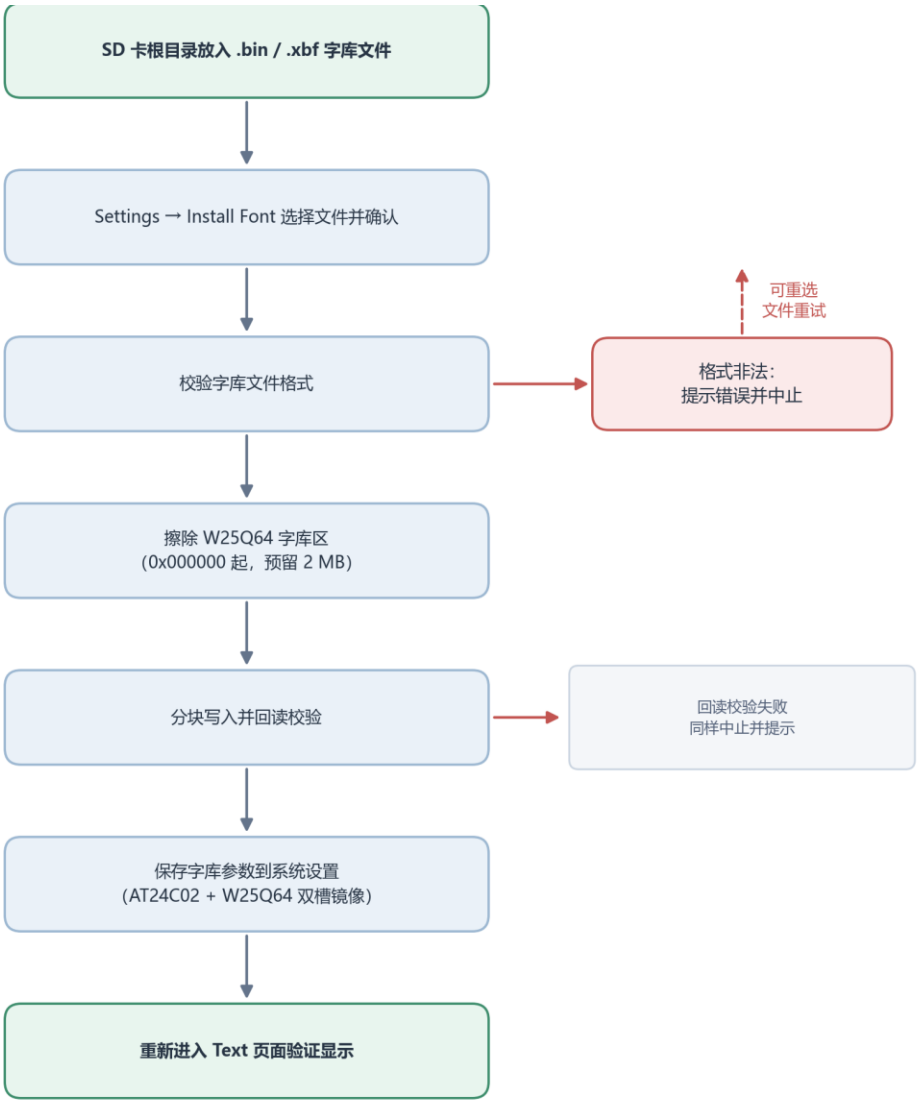


图 11　字库安装流程：校验、擦写、回读验证与设置保存全部完成后才会提示成功。

6.3　USB MSC 与升级

Install Update 固定查找 0:/update.bin。执行前必须在 Settings 关闭 USB MSC，否则页面提示 Disable USB MSC before update。固件会检查文件大小和向量表，将文件复制到 W25Q64 暂存区并做 CRC 校验，写入升级就绪记录后复位；BootLoader 随后备份当前 App、擦写应用区、编程并校验，失败时尝试从备份恢复。

> **升级警告** 升级期间不要断电、拔出 SD 卡或按复位键。建议使用稳定供电，确认 update.bin 是针对本机分区布局构建的完整应用镜像。

升级文件与流程的技术约束：

- update.bin 必须放在 SD 卡根目录；固件先检查大小（应用区上限 960 KB）与中断向量表合法性，再计算 CRC32。
- 校验通过后写入 W25Q64 暂存区，并在 EEPROM 写入升级记录（双槽容错），重启后由 BootLoader 完成烧写。
- 任意时刻断电都是安全的：BootLoader 上电读取记录，未完成的升级会被继续或回滚（见 14.1 节）。

第二部分　二次开发指导

7　UART Shell 命令

Shell 基于 Letter Shell v3.1.2，通过 UART1 工作；只有 Debug 和 Debug Shell 同时启用时才会创建。命令是否可见还受编译配置和运行时设置影响。默认提示符类似 admin:/$。

命令	参数	用途
help	[cmd]	查看帮助或单个命令。
users / cmds / vars	无	列出用户、命令和变量。
keys	无	查看按键状态。
cd / ls	<dir> / 无	FatFs 目录切换和列目录。
tasks	无	列出任务状态。
taskkill	<name> force	请求/强制结束指定任务。
reset	无	复位 MCU。
hexdump	<addr> <len>	查看内存，len 范围 1~65536。
sysstats	无	CPU、CCM/SRAMEX 堆使用率。
memfrag	无	各池空洞、最大连续块和碎片率。
videostat	[r]	查看/清零视频统计。

命令	参数	用途
audiostat	[r]	查看/清零音频统计。
sdinfo	无	显示 SD 卡信息。
clear	无	清屏。

7.1 诊断命令使用建议

诊断命令的价值在于“对比”：先记录基线，再对比场景中的变化。推荐的使用模式：

- 开机空载先执行 sysstats 与 memfrag 记录基线；进入场景（播放/NES/AXF）后再执行一次，差值即场景净消耗。
- 退出场景后复查各池占用是否回到基线，判断是否存在资源泄漏。
- videostat / audiostat 需在播放中执行才有意义；可先执行 videostat r 清零累计计数，再连续采样三次看趋势。
- 内存问题看 memfrag 的碎片率、最大连续块与分配失败计数（失败计数非零必须处理）；存储瓶颈看 videostat 的 io ticks 与 read_err；音频问题看 underrun 与 collide。

8 Shell 命令导出

工程启用了 SHELL_USING_CMD_EXPORT=1。使用 SHELL_EXPORT_CMD() 将函数放入 ARMCLANG 链接器识别的命令段，Shell 启动时自动发现，无需手动维护集中注册表。自定义命令应保持短小、避免阻塞 GUI/音频任务，并使用项目统一类型。

```
#include "shell.h"
#include "sys.h"

static int cmd_mempeek(int argc, char **argv)
{
    (void)argc; (void)argv;
    u32 value = *(volatile u32 *)0x20000000U;
    shellPrintEndLine("value=0x%08lX", (unsigned long)value);
    return 0;
}
SHELL_EXPORT_CMD(SHELL_CMD_PERMISSION(0) | SHELL_CMD_TYPE(SHELL_TYPE_CMD_FUNC),
                 mempeek, cmd_mempeek, "peek internal SRAM";
```

调试安全 hexdump、taskkill 和自定义内存命令可影响实时任务或外设状态。生产固件建议关闭 Debug/Shell，并避免在 DMA 活跃期间读取或修改缓冲区。

9 固件 API 使用约定

应用内部模块遵循 CMSIS-RTOS V2，使用 osThreadNew、osMutexNew、osSemaphoreNew 等接口，不直接调用原生 FreeRTOS API。线程栈从 CCM（SRAMCCM）通过 sys_thread_stack_alloc() 获取，临时线程退出后由父上下文回收。

API/模块	使用要点
sys_malloc / sys_free	按用途选择 SRAMCCM、SRAMEX；DMA 源/目标不能来自 CCM。
sys_malloc_aligned	DMA 缓冲和条带缓冲使用 32 B 对齐版本，并在退出时配对释放。
Page_Create / Page_Show / Page_Back	GUI 页面生命周期；页面历史栈深度 8，退出时 Page_Delete。
lvgl_stop_ui / lvgl_start_ui	媒体/AXF 访问 LCD 前后使用握手；等待 LVGL_UI_FLAG_PAUSED，不是简单布尔开关。
audio_card_open_prepare / audio_card_write	I2S3 DMA 声卡的准备和写入接口，音频源通过回调推送。
GUI_Theme_Color	页面配色从主题 token 获取，避免硬编码颜色。

CubeMX 安全区 主固件和 BootLoader 都由 CubeMX 管理。自定义代码放在 USER CODE BEGIN/END 之间，避免重新生成时丢失。新增 C 文件还需加入对应 Keil 工程组。

9.1 内存分配与线程创建规范

两个内存池由统一分配器管理，选错池是最常见的移植错误：DMA 无法访问 CCM，任务栈也不能用业务堆代替。选型决策流程如下：

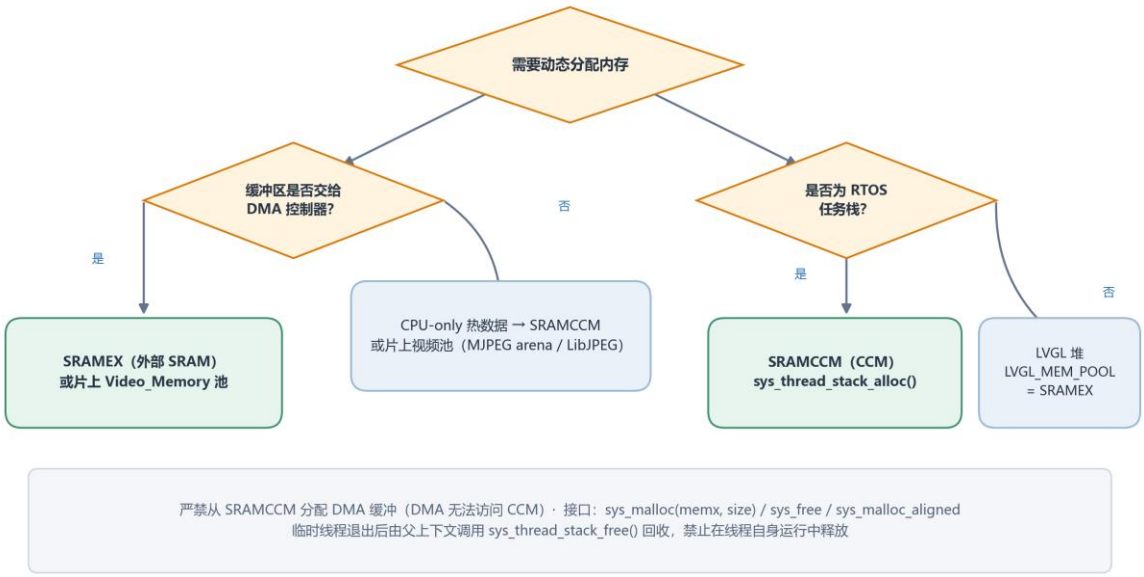


图 12　内存池选型决策：按"DMA / 任务栈 / CPU 热数据"三分。

线程一律使用 CMSIS-RTOS V2 创建，栈通过 sys_thread_stack_alloc() 取自 CCM（SRAMCCM）：

```
osThreadAttr_t attr = {0};
attr.name = "worker";
attr.stack_size = 2048;
attr.stack_mem = sys_thread_stack_alloc(attr.stack_size); /* 栈取自 CCM */
attr.priority = osPriorityNormal;
osThreadId_t tid = osThreadNew(worker_entry, NULL, &attr);
/* 线程退出后，由父上下文调用 sys_thread_stack_free(attr.stack_mem) 回收；
   禁止在线程自身仍在运行时释放其栈 */
```

10　AXF API 使用

外部 AXF 工程加入 ELF Loader/common、ELF Loader 和 EXTERN API 头文件路径，并包含 stargamer_axf_api.h。入口函数使用 DL_EXPORT 导出；像素参数为主机端 uint16_t RGB565，宿主会转换线序并使用 DMA 安全暂存。

```
#include "stargamer_axf_api.h"

DL_EXPORT int main(int argc, char *argv[])
{
    (void)argc; (void)argv;
    if (SG_LCD_Acquire() != SG_API_OK) return -1;
    SG_LCD_Clear(0x0000U);
    while (SG_ShouldExit() == 0U) {
        u32 keys = SG_KeyGetState();
        (void)keys;
        SG_AXF_Delay(10U);
```

```
    }
    SG_LCD_Release();
    return 0;
}
```

函数	作用
SG_LCD_Acquire / Release	获取/释放显示所有权，必须成对调用。
SG_LCD_Clear / DrawPixel / FillRect / BlitRGB565	基础绘图。
SG_LCD_GetWidth / GetHeight	读取当前 320×240 画布尺寸。
SG_KeyGetState / RawState	读取已适配或原始按键位图。
SG_ShouldExit	轮询宿主退出请求；PB7 映射为 EXIT。
SG_AXF_Delay	应用节拍延时。

SG_KEY_* 位掩码依次为 UP bit0、LEFT bit1、RIGHT bit2、DOWN bit3、OK bit4、A bit5、EXIT bit6、B bit7。Star Gamer ELF 和 Star Gamer ELF - 2048 是可直接参考的 Keil 工程；构建后把 Objects 目录的 AXF 复制到 SD 卡根目录，可用 Tests/AXF_Loader_Host/validate_axf.py 做主机端兼容性检查。

10.1 SVC 调用机制与资源模型

AXF 应用不直接链接 libc，也不直接触碰外设：所有跨边界调用都经 SVC 进入宿主。通道有两条：

- C 标准库：memcpy / strlen / snprintf 等 22 个函数经 SVC 8 查宿主向量表（dl_vector.c），AXF 内的调用点在重定位时被指向统一的 SVC 桩。
- 设备 API：stargamer_axf_api.h 中的 SG_* 内联函数触发 SVC，落到宿主 ExternAPI_* 实现；LCD 系列函数内部先 Acquire/Release 显示所有权，AXF 无法绕过 LVGL 互斥直接写屏。
- 加载段与运行时内存由宿主从 SRAMEX 分配，随卸载整体回收；应用退出后不残留任何分配。
- 构建产物建议先在 PC 端运行 Tests/AXF_Loader_Host/validate_axf.py 预检 ELF 兼容性。

第三部分　系统架构

11　启动链路与内存/存储框架

应用由 Core/Src/main.c 进入，初始化 HAL、时钟、DMA、FSMC、SDIO、UART、SPI、I2S、CRC、RNG 和定时器，然后启动 CMSIS-RTOS。defaultTask 调用 System/firmware_start.c 的 init()，建立内存池、挂载 SD、注册资源和 GUI 页面。

区域	地址/容量	主要用途
普通主 SRAM	0x20000000..0x2000BFFF / 48 KB	.data/.bss、MSP、C heap、FreeRTOS 控制块、NES LCD stripe 静态缓冲。
片上 Video_Memory	0x2000C000..0x2001FFFF / 80 KB	VideoTask 栈、FIL、AVI 上下文、JPEG source、LibJPEG 上下文 + 38 KB arena、RGB 行缓冲、LCD DMA 条带。
SRAMCCM	0x10000000 起 / 64 KB	RTOS 任务栈（defaultTask 等）、CPU-only 热数据；严禁 DMA。
SRAMEX	0x68000000 起 / 960 KB	LVGL 堆/帧缓冲、PCM FIFO、I2S DMA、GUI 上下文。
W25Q64	8 MB SPI NOR	字库 2 MB、Memory Dump、升级暂存/备份各 1 MB、设置镜像。
AT24C02	256 B I2C EEPROM	设置与 IAP 双槽记录。

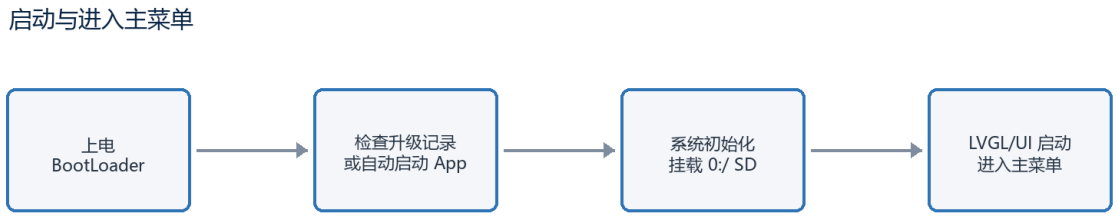


图 13　内存和启动流程的入口关系（与图 2 相同流程，便于架构章节查阅）。

内存选择原则 DMA 缓冲必须走 SRAMEX 或片上 Video_Memory；CCM 只能放 CPU 访问的数据。视频池分配失败会中止视频启动，使用 sysstats/memfrag 可定位容量和碎片问题。

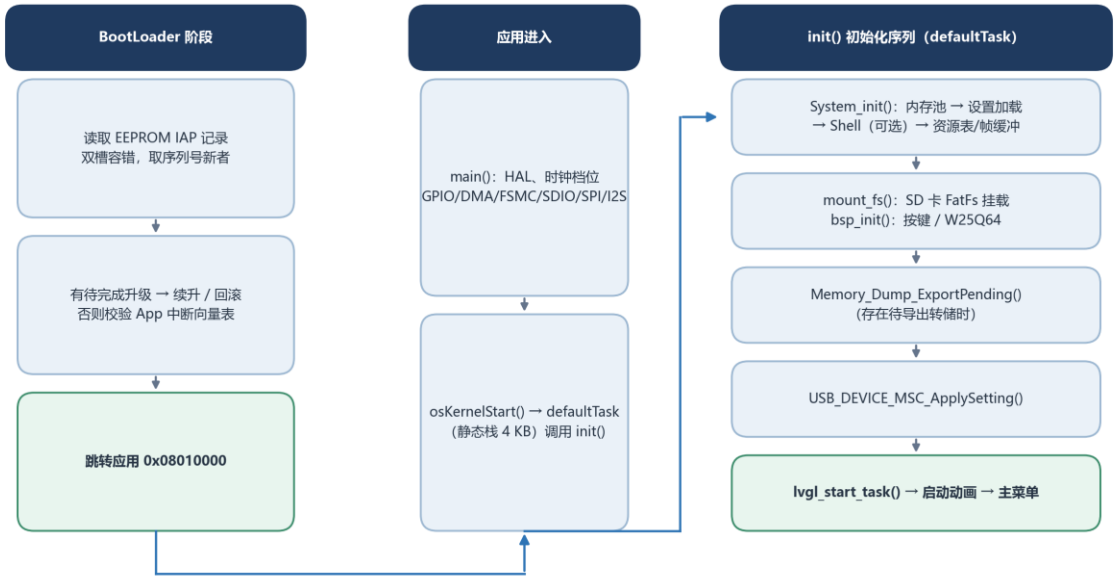


图 14　从上电到主菜单的完整启动链路：BootLoader 交权后，应用按固定顺序初始化。

应用侧初始化顺序有明确依赖：内存池先于一切动态分配；设置加载先于 GUI（主题、字库、时钟档位都来自设置）；文件系统挂载先于任何页面浏览。某一步失败时，错误经 Shell 或启动页提示，不会带着错误状态继续初始化。

11.1　存储介质映射

固件同时管理三块存储介质，用途划分固定，二次开发时不得挪用他区：



图15　内部Flash、W25Q64 与AT24C02 的分区映射。

- 内部 Flash 低 64 KB 属于 BootLoader，应用从 0x08010000 链接与执行。
- W25Q64 各分区边界由 IAP_Protocol.h 宏定义并有编译期断言保护；暂存区与备份区仅 IAP 使用。
- 设置写入走“双槽 + 序列号”协议：每次写入交替使用两个槽并递增序列号，读取取最新有效槽，写入中途掉电不影响旧数据。直接读写设置区会破坏该协议。

12　SDIO 防御性队列

SDIO 使用 DMA2_Stream3（RX）和 DMA2_Stream6（TX），FatFs 通过 512 B 对齐 scratch buffer 处理不对齐读写。底层 I/O 由请求队列串行化，给解析器、JPEG 和 PCM 供给明确的 owner/op 状态；忙时不会让多个任务同时触碰同一 FIL 或 DMA 寄存器。

- 请求包含方向、文件偏移、长度和完成结果，完成后由拥有者消费。
- DMA 缓冲位于可访问外部 SRAM，避免 CCM 不可 DMA 的硬件限制。
- videostat 中的 io active owner、io ticks、read_err 可用于判断 SDIO 是否成为瓶颈。
- FatFs/SDIO 日志默认关闭，需要时在 Settings → Debug 开启并重启。

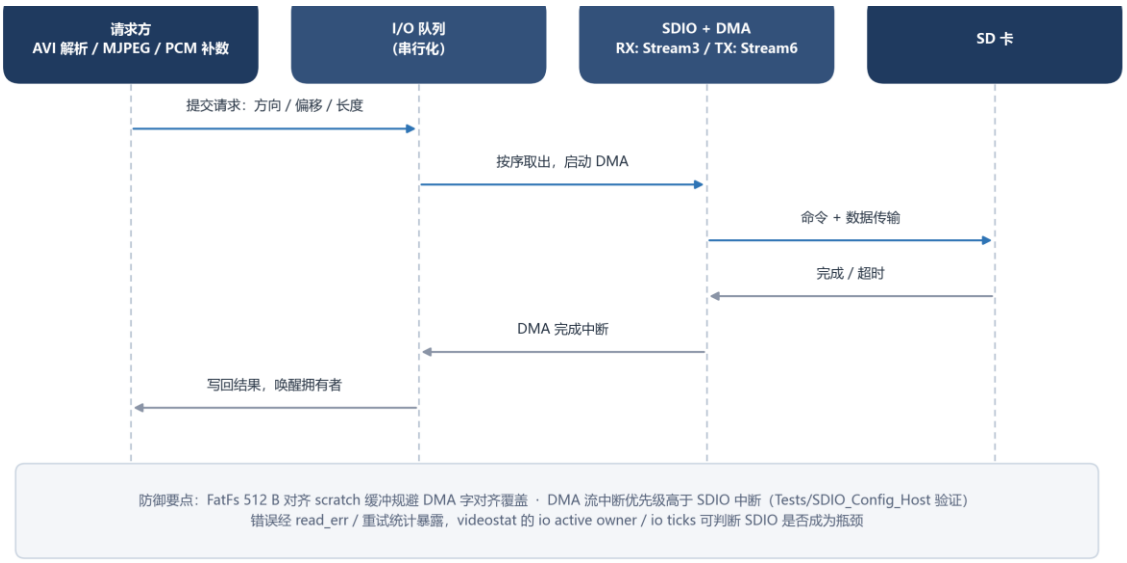


图 16　SDIO 防御性 I/O 队列：请求串行化，拥有者提交并消费结果。

队列把“谁在用 SD 卡”变成显式状态：每个请求绑定拥有者（AVI 解析器、JPEG 管线或 PCM 补数），完成结果由拥有者消费，任何时刻只有一个 DMA 传输在途，从机制上避免多模块并发读卡冲突。拥有者线程等待完成信号并带超时，错误统一记入 read_err 统计，可经 videostat 观察。

13　GUI 管理器与 LVGL-LCD 握手

GUI_Manage 以页面 ID 统一管理页面创建、显示、返回和删除，页面历史栈深度为 8。GUI_Task 负责页面编排，独立 lvgl_timer_task 每 1 ms 调用 lv_timer_handler()。页面上下文优先放 SRAMEX，避免占满普通主 SRAM。

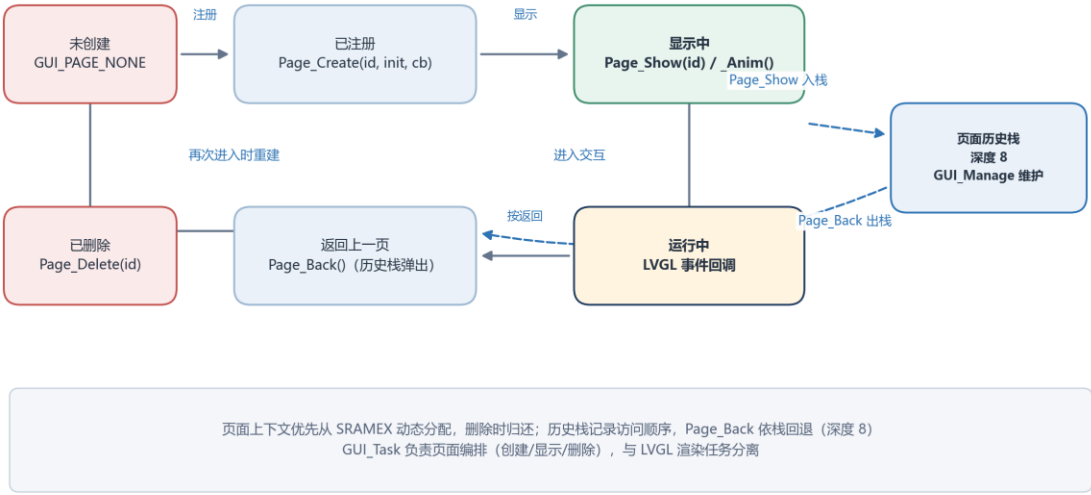


图 17 页面生命周期与历史栈：创建、显示、回退、销毁由 GUI_Manage 统一编排。

线程分工：GUI_Task 以 4096 字节栈运行于 osPriorityAboveNormal，负责页面编排；LVGL 渲染独立成任务（8192 字节栈，osPriorityNormal，每 1 ms 调用 lv_timer_handler）。页面配色统一经 GUI_Theme 获取（深/浅两套主题、20+ 颜色 token），页面代码不写死颜色值。

13.1 显示所有权握手

视频、NES 和 AXF 进入独占显示路径时，先调用 lvgl_stop_ui()：请求暂停、等待 LVGL 任务设置 LVGL_UI_FLAG_PAUSED，再操作 SPI1/LCD。完成后调用 lvgl_start_ui()；监督线程异常收尾可调用 lvgl_recover_ui()。这套握手用于保证 GUI 刷新、DMA 和媒体输出不会并发改写 LCD。

阶段	责任方	关键动作
普通页面	LVGL/UI	按键输入、控件刷新、主题 token。
独占显示	Video/NES/AXF	请求暂停 LVGL，获得 LCD 所有权。
退出/恢复	页面或 supervisor	释放缓冲、恢复 LVGL，必要时走 recover。

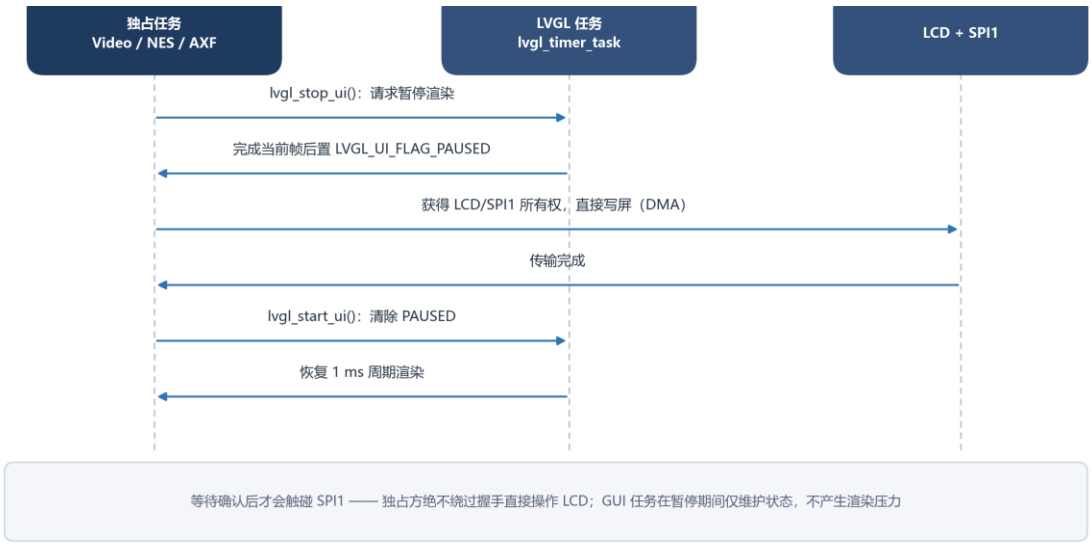


图 18　LVGL 与独占任务的显示所有权握手时序。

14　BootLoader + IAP

Flash 低 64 KB（0x08000000..0x0800FFFF，扇区 0–3）属于 BootLoader；应用从 0x08010000 开始，最大 960 KB。IAP 使用 SD 的 0:/update.bin、W25Q64 暂存区 0x00600000、备份区 0x00700000 和 EEPROM 双槽序列号记录实现断电容错。

固件升级与失败恢复



图 19　IAP 升级流程。

BootLoader 菜单	作用	按键
RESTORE BACKUP	从 W25Q64 备份恢复 App。	Down/Up 选择，OK 执行，PB7 取消。
INSTALL UPDATE	安装暂存区镜像。	OK 执行。
BACKUP APP	把当前 App 备份到 W25Q64。	OK 执行。

BootLoader 菜单	作用	按键
BOOT APP	跳转有效应用。	OK 执行。

14.1 升级状态机与断电恢复

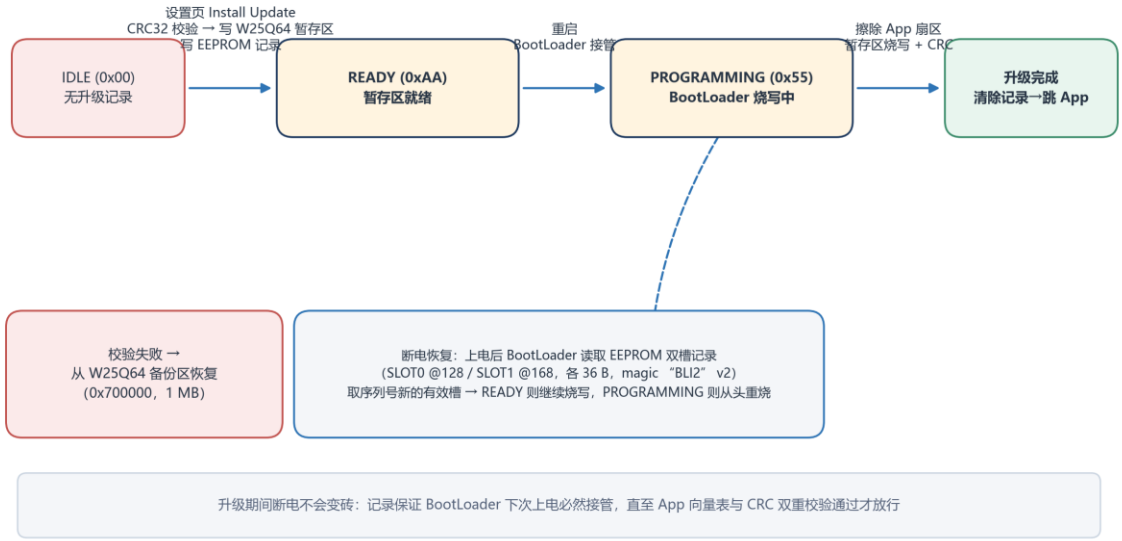


图 20 IAP 升级状态机：EEPROM 记录驱动，BootLoader 每次上电接管未完成的升级。

升级记录的关键设计：

- 记录共 36 字节，magic"BLI2"、版本 2，存于 EEPROM 双槽：SLOT0 @128、SLOT1 @168；写入按序列号递增交替，读取取序列号新的有效槽。
- 状态码：0xAA 暂存区就绪（READY）、0x55 烧写中（PROGRAMMING）、0xBB 请求恢复备份、0xCC 备份有效。
- 记录结构由 IAP_RECORD_MEMBERS 宏定义，App 与 BootLoader 共享 IAP_Protocol.h；扩展字段必须两侧同步，并保持 IAP_RECORD_SIZE 一致。

15 AXF 加载器架构

ELF Loader 由 dl_lib（装载/卸载/符号查找）、dl_elf（格式结构）、dl_arch（Cortex-M 重定位）、dl_port（文件/内存抽象）和 dl_vector（宿主函数向量）组成。应用通过 SVC 向量调用宿主的 C 库、LCD、按键和延时服务；GUI_Axf 页面负责文件选择、线程栈分配、运行监控和退出回收。

- 宿主把 AXF 看作可回收的运行时资源，线程退出后由 supervisor 回收栈和加载段。
- LCD API 通过 acquire/release 与 LVGL 互斥，避免 AXF 直接绕过显示所有权。
- 动态应用不应依赖未导出的 libc 符号；在构建后运行 validate_axf.py。

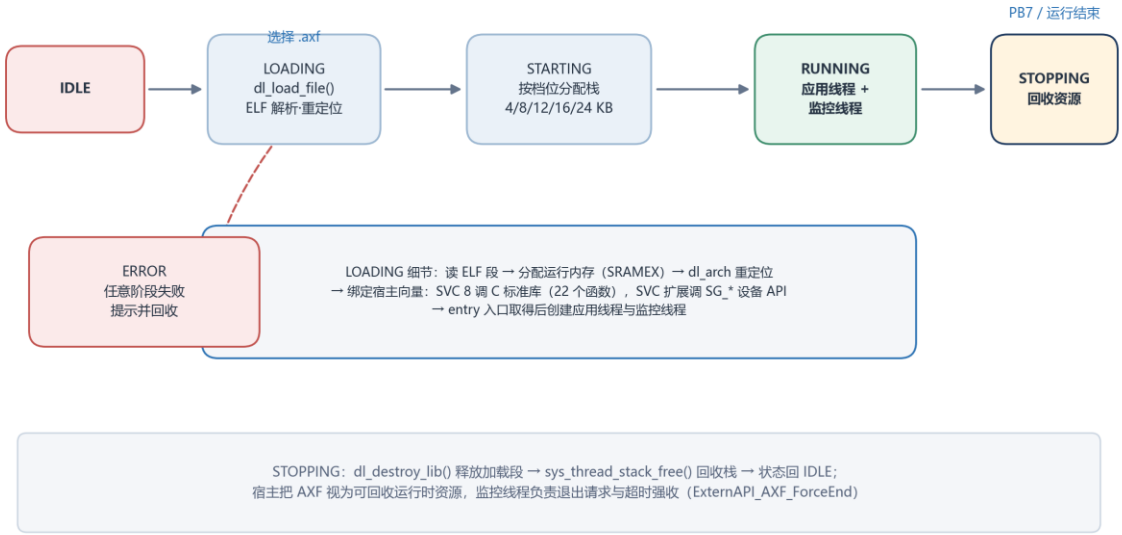


图 21　AXF 加载器状态机与 SVC 通道：加载、运行、回收全程由宿主监督。

加载器组件分工：dl_lib 负责装载、符号查找与销毁；dl_elf.h 定义 ELF 结构解析；dl_arch 处理 Cortex-M 重定位；dl_port 抽象文件与内存；dl_vector / dl_vector_api 组成宿主函数向量表。宿主侧以“会话”管理一次运行（ExternAPI_AXF_Begin / End / ForceEnd），LCD API 通过 acquire/release 与 LVGL 互斥。用户 API 编号定义于 dl_user_api.h，宿主与 demo 工程（Star Gamer ELF）两侧保持同步。

16　视频播放器、LibJPEG 与 LCD 流水线

Video.c 组织 AVI 状态机、音视频同步、Seek 和丢帧策略；AVI_Parser.c 负责 RIFF/little-endian 解析、流索引和 chunk 边界容错；Video_MJPEG.c 使用 LibJPEG 流式输入和片上视频池 arena；Video_PCM.c 维护 32 KB kfifo、4 KB I/O buffer 与 I2S DMA 双半缓冲。

AVI 播放数据流

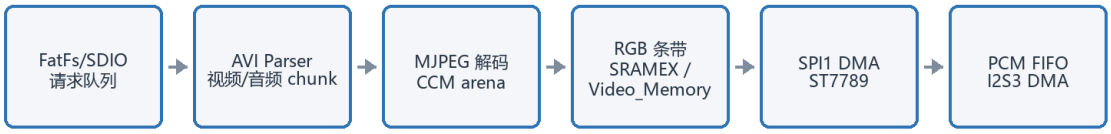


图 22　AVI 播放流水线：文件读取、解码、显示和音频补数由不同缓冲区协作。

当前 MJPEG 条带高度为 20 行，视频源缓冲 8 KB，JPEG arena 38 KB；PCM FIFO 预缓冲水位约 24 KB，低水位 16 KB、高水位 28 KB。音频时钟驱动帧 deadline，遇到解码超时优先丢弃过期画面而不主动打断音频。

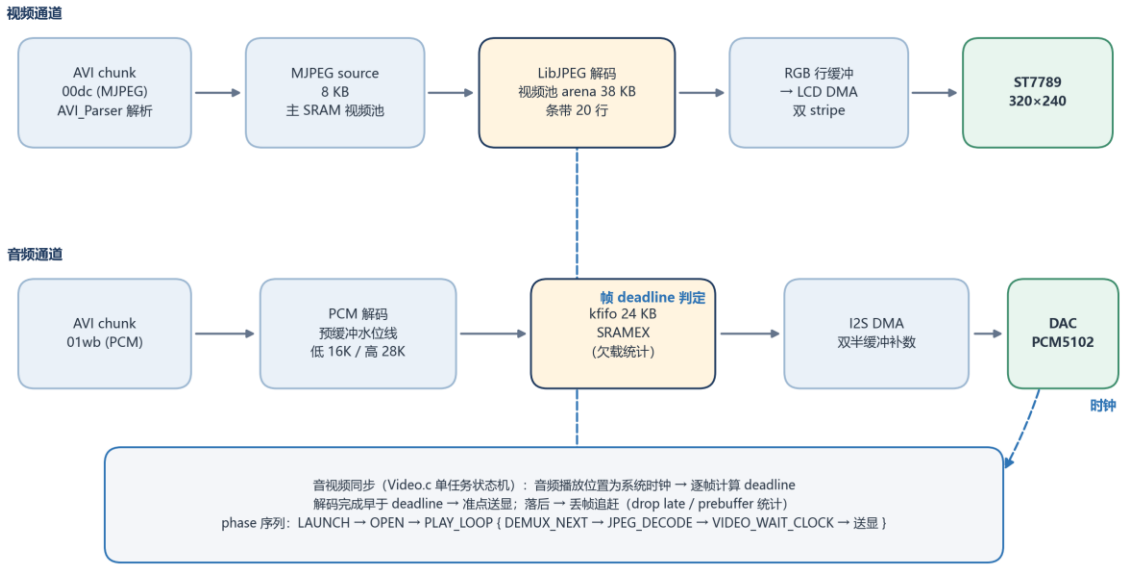


图 23　AVI 播放双通道流水线：音频时钟驱动帧 deadline，解码与 LCD 传输重叠执行。

播放循环由音频时钟驱动：每个视频帧以“音频已播放帧数”换算 deadline，解码提前完成则等待送显，落后则丢帧追赶（计入 drop late）。LCD 写入走双 stripe DMA，与 JPEG 解码在片上 SRAM 中的计算重叠执行；偶发的大帧解码超时表现为一次性连丢数帧，而声音始终连续——音频 FIFO 的 24 KB 余量提供了缓冲。

第四部分　性能、诊断与附录

17　视频性能参考报告

以下数据来自当前工程的 videostat 记录，是开发和选片时的工程参考，不是对所有 SD 卡、文件和温度条件的保证。判断故障时，jpeg_err、lcd_dma_timeout、dma_err、i2s_err 和 read_err 比单独的 dropped 更重要。

样例	媒体参数	观测值	结论
23.976 FPS / 2 Mbps	320×240，MJPG，22050 Hz/双声道 PCM，4171 帧	early：displayed 414 / dropped 16；后段 displayed 2407 / dropped 46；jpeg_err=0，lcd_dma_timeout=0，collide=0；underrun=9	低码率下画面基本跟随时钟，偶发晚帧但没有解码或 DMA 错误。
25 FPS / 10 Mbps	320×240，MJPG，22050 Hz/双声道 PCM，5329 帧	early：displayed 721 / dropped 156，jpeg_max=69；重载段 displayed 1226 / dropped 293，jpeg_max=273，underrun=27；dma_err=0	高码率下 JPEG 峰值和晚帧明显增加，系统通过丢帧保持音频连续。

17.1　如何读 videostat

- video ... fps/frame_us：容器声明的帧率和目标帧周期。
- decoded/displayed/dropped：解码、送显和为追赶音频时钟而丢弃的帧。
- timing jpeg_max/lcd_dma_timeout：JPEG 最长耗时和 LCD DMA 超时；超时非零应优先检查负载、SPI 和缓冲。
- pcm fifo/underrun、audio collide/late：音频 FIFO 水位、欠载和 DMA 碰撞。
- jpeg_arena_peak/arena_fail、vmem ... fail：视频池 arena 峰值与视频池分配失败计数。

建议在同一文件的开头、中段和高负载段各执行一次 videostat，必要时先执行 videostat r 清零，以观察趋势而不是只看单个瞬时值。

17.2 两组实测数据对比

下表摘取两组样片各三次 videostat 采样中的累计值（同机、同卡），便于直观感受码率的影响：

指标	23.976 fps / 2 Mbps	25 fps / 10 Mbps	解读
累计丢帧	46 帧 / 送显 2407（约 1.9%）	293 帧 / 送显 1226（约 19%）	高码率下以丢帧换实时
drop late	40	287	绝大多数丢帧源于追赶音频时钟
jpeg_max 单帧解码	102 ms	69 ms（单次峰值 273 ms）	个别大帧仍会瞬时阻塞流水线
PCM underrun	9（开局预缓冲后不再增加）	16 → 27 缓增	音频 FIFO 始终有余量垫底
audio DMA collide	0	0	本批次固件未复现碰撞
SD 读取累计	34.3 MB / 9390 次（进度 60%）	31.0 MB / 6575 次（进度 26%）	高码率单位时间吞吐显著更高

两组数据说明：2 Mbps 样片全程音画同步、几乎无可感丢帧；10 Mbps 样片画面持续以丢帧保持实时，但声音始终连续、无 DMA 碰撞。瓶颈在 JPEG 解码与 SD 吞吐，音频通路余量充足。

17.3 选片与转码建议

- 编码：MJPEG + PCM，分辨率 ≤320×240；超过上限的文件会被解析器直接拒绝。
- 帧率与码率：23.976 或 25 fps；视频码率 ≤2 Mbps 可获得全帧率体验，10 Mbps 可播放但持续丢帧。
- 音频：PCM 22050 Hz / 16 bit，建议立体声；音频参数异常会导致预缓冲失败而无法开播。
- 视频来源建议自行转码压制，网络片源直接拷贝通常码率远超预算，只能以丢帧模式播放。

18 故障排查与诊断

现象	优先检查	处理建议
Text 提示 Install font first	W25Q64 字库区和设置记录	重新安装 .bin/.xbf，确认回读验证成功；检查 W25Q64 ID。
升级页面拒绝执行	USB MSC 是否开启、update.bin 路径	关闭 USB MSC；文件必须在 0:/update.bin。
视频卡顿但有声音	dropped、jpeg_max、underrun	降低 MJPEG 码率/帧率；检查 SD 卡读速和 SRAMEX 碎片。
视频黑屏或 UI 不恢复	lvgl_stop/start 日志、LCD DMA	退出后等待握手完成；必要时用 lvgl_recover_ui() 的监督路径。
NES memory error	sysstats/memfrag、SRAMEX	退出其他媒体/AXF，减少碎片后重试。
AXF 无法退出	是否轮询 SG_ShouldExit、是否 Release LCD	修复应用循环；宿主 100 ms 后会强制收尾。
HardFault	LCD 上寄存器转储	记录 PC/LR/CFSR/HFSR；若启用 Memory Dump，可导出 0:/hardfault_dump.bin。

18.1 HardFault 与内存转储

发生 HardFault 时无需调试器：固件内嵌的 Handler 以独立重初始化的 SPI1 直接驱动 LCD，在屏幕上显示 R0–R3、R12、LR、PC、xPSR，以及 CFSR / HFSR / DFSR / AFSR / MMFAR / BFAR 和 MSP / PSP（5×7 字体，320×240）。由 CFSR 可区分总线/用法/存储器错误，PC 与出错地址可直接在 .map 文件中定位函数。

若开启了内存转储，系统会按设置的区域开关把 CCM、主 SRAM、外部 SRAM 的内存映像写入 W25Q64 转储分区（3.5 MB，FNV-1a 校验，分区边界有编译期检查）；下次开机经 Shell 命令导出到 SD 卡根目录 hardfault_dump.bin，配合同版本 .map 文件回溯现场。

附录 A　构建、目录与支持范围

主固件工程为 MDK-ARM/Star Gamer.uvprojx，BootLoader 工程为 BootLoader/StarGamerBootLoader/MDK-ARM/StarGamerBootLoader.uvprojx，均使用 Keil MDK-ARM V5.32 / ARMCLANG，按 F7 构建。没有 Makefile 或 CMake。F8 下载，Project → Clean Targets 清理。

目录	用途
Core / Drivers / BSP	CubeMX 入口、HAL、LCD、按键、W25Q64、I2S。
System / GUI	内存池、系统状态、HardFault/Memory Dump、页面管理器。
Media	Player、AVI Parser、MJPEG、PCM。
Middlewares	FreeRTOS、FatFs、LVGL、NES、Arduboy、LibJPEG、Helix MP3。
ELF Loader / EXTERN API	AXF 装载器和宿主服务。
Tests / Tools / Docs	主机验证脚本、PC 工具、设计和用户文档。

版本边界 BMP 图像库目前是占位模块；视频播放器按当前实现只保证 MJPEG + PCM 路径；Arduboy 游戏来自固件内置注册表。新增格式或页面后，应同步更新本手册的支持表和按键说明。

文档编写依据：当前源代码、设置页面可见文本、IAP/AXF API 头文件、Shell 命令实现和 Docs/images 实机图片。